SEMANA 2 · UNIDAD 2

Modelos de Aprendizaje Supervisado

26 sep – 2 oct 2026 · Clase sincrónica: sábado 26/09 · tutoría 3 h + laboratorio 1.5 h

48 h: 4.5 ACD · 18 APE · 25.5 AA

RDA1 · CG-G1 · CE-G1 · Bibliografía: BB1 · BB3 · BB4 · BB5 · BC3 · BC7

MANUAL M2 · TEORÍA DE LA SEMANA

Modelos lineales, SVM, k vecinos, árboles y comparación honesta

Lectura base de la semana 2 (2.5–3 h). Cada sección termina con un enlace **Practica esto →** al nivel exacto del ejercicio o a la animación que la ilustra. Las cifras citadas salen de la ejecución real de los notebooks.

CONTENIDO

§1 Regresión lineal
§2 Regularización
§3 Regresión logística
§4 Calibración de probabilidades
§5 Máquinas de soporte vectorial
§6 k vecinos más cercanos (KNN)
§7 Árboles de decisión
§8 Bosques aleatorios y boosting
§9 Comparar modelos con honestidad
§10 Autoevaluación y lecturas

§1 Regresión lineal

La regresión lineal modela la respuesta como una combinación lineal de las variables más un intercepto. Con n observaciones y p variables, y una columna de unos en la matriz X para el intercepto:

$$\hat{y} = X\beta, \qquad \hat{\beta} = \arg\min_{\beta} \frac{1}{n}\|y - X\beta\|^2$$

La pérdida cuadrática (el ECM) es un **cuenco convexo** en el espacio de los coeficientes: tiene un único mínimo. Hay dos formas de llegar a él.

- **Ecuación normal** (solución cerrada): igualar el gradiente a cero da $X^\top X\,\hat{\beta} = X^\top y$, es decir $\hat{\beta} = (X^\top X)^{-1} X^\top y$. En la práctica no se invierte la matriz: se resuelve el sistema (`np.linalg.solve`) o se usa una descomposición QR o SVD, que es lo que hace `LinearRegression`. Su costo crece con p^3.

- **Descenso de gradiente** (iterativo): $\nabla L = \frac{2}{n} X^\top (X\beta - y)$ y cada paso hace $\beta \leftarrow \beta - \eta \nabla L$. Escala a millones de filas, sobre todo en su versión **estocástica** por mini-lotes (`SGDRegressor`).

La **tasa de aprendizaje** η decide todo: si es muy pequeña el método es lento; si supera $2/\lambda_{\max}$ (el mayor valor propio del hessiano $\frac{2}{n} X^\top X$) cada paso se pasa del mínimo y el entrenamiento **diverge**. Las variables en escalas muy distintas estiran el cuenco hasta volverlo un valle angosto: la tasa estable queda limitada por la dirección empinada y el avance por la dirección plana es lentísimo. Por eso se **estandariza** (media 0, desviación 1) antes de entrenar.

> **Interpretar coeficientes**
> β_j es el cambio esperado en y cuando x_j aumenta en una unidad **y las demás variables quedan fijas**. Con variables estandarizadas, el cambio es por una desviación estándar y los coeficientes se pueden comparar entre sí. Con variables correlacionadas los coeficientes individuales se vuelven inestables, aunque las predicciones sigan siendo buenas. Los supuestos clásicos (linealidad, errores independientes con varianza constante) importan para la inferencia; para predecir, lo decisivo es validar fuera de la muestra.

Python

```python
import numpy as np

Xb = np.column_stack([np.ones(len(X)), X])      # intercepto
beta = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)     # ecuación normal

beta_gd = np.zeros(Xb.shape[1])                 # gradiente (X estandarizada)
for _ in range(500):
    grad = 2 / len(y) * Xb.T @ (Xb @ beta_gd - y)
    beta_gd -= 0.1 * grad
```

Practica esto → E2.1 · Nivel 1

(ecuación normal y gradiente; con x sin escalar, η = 8·10⁻⁴ ya diverge) · **Animación A2.1**

§2 Regularización

Con muchas variables, variables correlacionadas o pocos datos, los mínimos cuadrados producen coeficientes grandes e inestables: ajustan el ruido. La **regularización** agrega a la pérdida un castigo por el tamaño de los coeficientes (nunca del intercepto):

$$\text{Ridge: } \|y - X\beta\|^2 + \alpha\|\beta\|_2^2 \qquad \text{Lasso: } \tfrac{1}{2n}\|y - X\beta\|^2 + \alpha\|\beta\|_1$$

- **Ridge (L2)** encoge todos los coeficientes hacia cero de forma suave, sin anular ninguno. Tiene solución cerrada: $\hat{\beta} = (X^\top X + \alpha I)^{-1} X^\top y$ con X e y centradas. Es muy útil con variables correlacionadas.
- **Lasso (L1)** anula coeficientes exactamente: **selecciona variables**. Geométricamente, la región $\|\beta\|_1 \leq t$ es un rombo cuyas esquinas están sobre los ejes, y los contornos elípticos de la pérdida suelen tocarlo primero en una esquina.
- **Elastic Net** combina ambas (`l1_ratio`): selecciona como Lasso pero, con variables correlacionadas, tiende a conservarlas en grupo.

Dos reglas prácticas. Primera: **estandarizar antes**, dentro del pipeline, porque la penalización trata a todos los coeficientes por igual y una variable medida en miles quedaría castigada de más. Segunda: α **se elige por validación cruzada** (`RidgeCV`, `LassoCV`); nótese que cada clase de scikit-learn escala α de forma distinta, así que un valor no se traslada de un modelo a otro.

> **Cuándo ayuda y cuándo no**
>
> En E2.1, con 6 variables de ruido y 200 filas, Lasso anula 5 de las 6 y conserva las dos reales. En California Housing (8 variables y 16 mil filas) la regularización no cambia el error (MAE ≈ 0.53 con y sin ella): hay muchos más datos que parámetros y el problema del modelo lineal es el **sesgo**, no la varianza. Lo que sí ayuda es reducir el sesgo: con interacciones de grado 2 (44 variables) el MAE baja a ≈ 0.48, y Lasso sirve para quedarse con las que importan (con α = 0.02 sobreviven 10 de 44).

Python

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV

ridge = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 30)))
lasso = make_pipeline(StandardScaler(),
                      LassoCV(alphas=np.logspace(-4, 0, 30), cv=5))
```

Practica esto → E2.1 · Niveles 2 a 4

(Ridge desde cero, ruta de Lasso en California Housing y variables polinómicas) ·

Animación A2.2

§3 Regresión logística

Para clasificar, la regresión logística calcula un puntaje lineal y lo transforma en probabilidad con la **sigmoide**:

$$z = w^\top x + b, \qquad p(y=1 \mid x) = \sigma(z) = \frac{1}{1+e^{-z}}$$

El puntaje z es proporcional a la distancia con signo a la frontera $w^\top x + b = 0$, que es un hiperplano: la regresión logística es un clasificador **lineal**. Los coeficientes se leen en *log-odds*: e^{w_j} es el factor por el que se multiplican las chances $p/(1-p)$ cuando x_j sube en una unidad.

Se entrena minimizando la **log-loss** (entropía cruzada), que castiga con fuerza la confianza equivocada: predecir 0.05 para un positivo cuesta $-\log 0.05 \approx 3.0$, mientras que acertar con 0.9 cuesta 0.11.

$$L(w,b) = -\frac{1}{n}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \qquad \nabla_w L = \frac{1}{n} X^\top (p - y)$$

- **Regularización:** scikit-learn aplica L2 por defecto con $C = 1$, donde C es el *inverso* de la fuerza ($C = 1/\lambda$): menos C, más regularización. Sin regularización y con clases separables, $\|w\|$ crece sin fin y las probabilidades se van a 0 y 1.
- **Multiclase:** la versión multinomial (softmax) reemplaza a la sigmoide; es la opción por defecto en scikit-learn.
- **Desbalance:** `class_weight="balanced"` pesa cada clase en proporción inversa a su frecuencia. Sube el recall con umbral 0.5, pero **desplaza las probabilidades**: en Bank Marketing (11.7 % positivos) la probabilidad media predicha pasa a 0.42. Ver §4.
- **Probabilidad no es decisión:** el umbral se elige por el costo de cada error, como en la semana 1.

> **Practica esto →** **E2.2 · Niveles 1 y 2** (sigmoide, log-loss y gradiente desde cero; efecto de `C` y `class_weight`) · **Animación A2.3**

§4 Calibración de probabilidades

Un modelo está **calibrado** si, entre todos los casos a los que asigna una probabilidad cercana a q, la fracción de positivos es q. Separar bien (AUC alto) no lo garantiza: cualquier transformación monótona de los puntajes conserva el orden, y por lo tanto la AUC, pero cambia su significado.

- **Diagrama de fiabilidad:** agrupa las predicciones en intervalos y grafica la probabilidad media predicha contra la frecuencia observada. Calibrado = sobre la diagonal. Una S por debajo a la derecha indica sobreconfianza (típico de SVM escaladas, boosting, Naive Bayes); una S invertida, subconfianza (típico de bosques aleatorios).
- **Brier score** $\frac{1}{n}\sum_i (p_i - y_i)^2$ y **ECE** (error de calibración esperado, la distancia media a la diagonal ponderada por casos).

- **Calibrar** con `CalibratedClassifierCV`, que aprende una transformación monótona sobre folds que el modelo no vio: **Platt** (`method="sigmoid"`) ajusta $p' = \sigma(a\,\mathrm{logit}(q) + b)$; la **isotónica** ajusta una función escalonada creciente, más flexible pero necesita más datos (regla práctica: más de 1000 casos de calibración).

> **Por qué importa: decidir con valor esperado**
> En E2.2, llamar a un cliente cuesta 1 y un depósito contratado deja 8: conviene llamar si $8p - 1 > 0$, es decir si $p > 0.125$. Con la logística calibrada esa regla da un beneficio de 3637 en la prueba; con el modelo balanceado sin calibrar, el mismo umbral hace llamar a 11 010 clientes y el beneficio es -490. Tras calibrar el modelo balanceado (Platt), vuelve a 3701. Las probabilidades se calibran siempre que se usen *como probabilidades*.

Python

```python
from sklearn.calibration import CalibratedClassifierCV, CalibrationDisplay

cal = CalibratedClassifierCV(modelo, method="sigmoid", cv=5).fit(X_train, y_train)
CalibrationDisplay.from_estimator(cal, X_test, y_test, n_bins=10)
```

Practica esto → E2.2 · Niveles 3 y 4

(Bank Marketing: fiabilidad, valor esperado, Platt frente a isotónica) · **Animación A2.4**

§5 Máquinas de soporte vectorial

Si dos clases son separables, hay infinitas rectas que las separan. La SVM elige la del **margen máximo**: la que deja la franja ("calle") más ancha sin puntos. Con etiquetas $y_i \in \{-1, +1\}$ y la frontera $w^\top x + b = 0$, el ancho de la calle es $2/\|w\|$, así que maximizarlo equivale a minimizar $\|w\|^2$.

El **margen blando** permite violaciones con un costo C y lleva a minimizar la **pérdida hinge** regularizada:

$$\min_{w,b} \; \frac{1}{2}\|w\|^2 + C \sum_i \max\left(0,\; 1 - y_i(w^\top x_i + b)\right)$$

- **C grande:** pocas violaciones, calle angosta, riesgo de sobreajuste. **C pequeño:** calle ancha, más errores de entrenamiento; muy pequeño lleva al subajuste.
- **Vectores de soporte:** en la formulación dual, $f(x) = \sum_i \alpha_i y_i K(x_i, x) + b$ con $0 \le \alpha_i \le C$. Solo los puntos sobre el margen o dentro de él tienen $\alpha_i > 0$; los demás no influyen en la

frontera.

- **Truco del kernel:** el dual solo usa productos $K(x_i, x_j)$. Un kernel calcula el producto interno en un espacio transformado sin construirlo: elevar $z = x_1^2 + x_2^2$ separa círculos concéntricos con un plano. El **RBF** $K = e^{-\gamma\|x-x'\|^2}$ equivale a un espacio de infinitas dimensiones.
- **gamma:** pequeño = cada punto influye lejos, frontera suave; grande = cada punto crea su isla, sobreajuste. C y γ se buscan juntos en una malla logarítmica.
- **Escalado obligatorio:** el RBF depende de distancias. Y el costo computacional del `SVC` con kernel crece más que cuadráticamente con n: en la ejecución de referencia de E2.3, multiplicar n por 8 multiplicó su tiempo por ≈ 28, frente a ≈ 5 de `LinearSVC`.

La SVM no produce probabilidades por sí misma: `SVC(probability=True)` aplica Platt con validación cruzada interna (más lento). Para rankear basta `decision_function`.

> **Practica esto → E2.3**
>
> (subgradiente de la hinge, malla C × gamma y detección de phishing: F1 0.916 del baseline lineal frente a 0.960 del RBF) ·
>
> **Animación A2.5**

§6 k vecinos más cercanos (KNN)

KNN es el modelo más simple de todos: **no aprende parámetros**. Guarda el conjunto de entrenamiento y, para un caso nuevo x, busca sus k vecinos más cercanos $N_k(x)$ y los deja votar. En clasificación, la probabilidad estimada de cada clase es la proporción de vecinos de esa clase; en regresión, la predicción es el promedio de sus valores:

$$\hat{p}(c \mid x) = \frac{1}{k} \sum_{i \in N_k(x)} 1[y_i = c] \qquad \hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

- **k controla la complejidad:** con $k = 1$ el modelo memoriza (exactitud de entrenamiento 100 %, frontera llena de islas: varianza alta); con k grande promedia vecindarios enormes y la frontera se aplana (sesgo alto). Es un hiperparámetro: se elige por validación cruzada y, en problemas binarios, impar para evitar empates. En la animación A2.11, con lunas ruidosas, la exactitud de validación pasa de 84 % con k = 1 a 90 % con k = 15 y vuelve a 84 % con k = 61.
- **La distancia lo es todo:** la euclidiana $\|x - x'\|_2$ es la habitual; la Manhattan $\sum_j |x_j - x'_j|$ pesa menos una sola diferencia grande. **Escalar es obligatorio**: si una variable está en una unidad mayor, domina la distancia y las demás dejan de contar. En E2.3, registrar una variable × 100 baja la exactitud de 0.895 a 0.792; el `StandardScaler` dentro del pipeline la recupera.
- **Pesos por distancia:** con `weights="distance"` cada vecino vota con peso $1/d$ y los cercanos pesan más. Consecuencia: la exactitud de entrenamiento es siempre 100 % (cada punto es su propio vecino, a distancia 0) y deja de servir para diagnosticar sobreajuste.

- **Costo invertido:** "entrenar" es guardar los datos, pero cada predicción calcula n distancias en d dimensiones; los árboles KD o *ball tree* aceleran la búsqueda cuando hay pocas variables. En phishing (E2.3), KNN con k = 21 y pesos 1/d llega a F1 0.949 frente a 0.960 de la SVM RBF, pero guarda los 8 291 sitios de entrenamiento, y el tiempo de predecir crece con n.
- **Maldición de la dimensión:** con muchas variables, todas las distancias se parecen. En la animación A2.11, al agregar 100 variables de ruido, el vecino más cercano pasa a estar al 0.75 de la distancia del más lejano (0.04 en 2D) y la exactitud cae de 90 % a 70 %. KNN necesita pocas variables relevantes: selección de variables o reducción de dimensión (PCA, semana 3).

```python
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

knn = make_pipeline(StandardScaler(), KNeighborsClassifier())
malla = {"kneighborsclassifier__n_neighbors": [1, 5, 11, 21, 41],
         "kneighborsclassifier__weights": ["uniform", "distance"]}
busqueda = GridSearchCV(knn, malla, cv=5, scoring="f1").fit(X_train, y_train)
```

KNN rara vez gana en datos tabulares grandes, pero es un **modelo de referencia** honesto y fácil de explicar ("este caso se parece a estos 21"), sirve para imputar faltantes (`KNNImputer`), y su idea de vecindario reaparece en DBSCAN y LOF (semana 3).

> **Practica esto → E2.3 · Niveles 1–4**
>
> (KNN desde cero con broadcasting, k por validación cruzada, escalado, pesos 1/d y comparación con la SVM en phishing) ·
>
> **Animación A2.11**

§7 Árboles de decisión

Un árbol divide el espacio con preguntas del tipo $x_j \leq t$. El algoritmo CART es **voraz**: en cada nodo prueba todos los umbrales de todas las variables y elige el corte que más reduce la impureza; luego repite en cada hijo. Cada hoja predice la clase mayoritaria (o la proporción) de sus datos.

$$\text{Gini} = 1 - \sum_k p_k^2 \qquad \text{Entropía} = -\sum_k p_k \log_2 p_k$$

$$\text{ganancia} = I(\text{padre}) - \left[\frac{n_I}{n} I(\text{izq.}) + \frac{n_D}{n} I(\text{der.})\right]$$

- **Ventajas:** reglas legibles, sin escalado, mezclan variables numéricas y categóricas, capturan interacciones.
- **Profundidad y sobreajuste:** sin límites, el árbol aísla cada punto: en credit-g el AUC de entrenamiento llega a 1.00 mientras el de validación cae de 0.68 (profundidad 4) a 0.60.
- **Parada y poda:** `max_depth`, `min_samples_leaf`, `min_impurity_decrease` y la poda por costo-complejidad `ccp_alpha`, que recorta ramas que aportan poco. Se eligen por validación cruzada.
- **Inestabilidad:** un pequeño cambio en la muestra puede cambiar el primer corte y todo el árbol. Es su punto débil y la razón de los ensambles.

> **Practica esto →** **E2.4 · Niveles 1 y 2** (Gini y mejor corte desde cero: `duration ≤ 17` en credit-g, igual que scikit-learn) · **Animación A2.6**

§8 Bosques aleatorios y boosting

Bagging y bosques aleatorios

Bagging entrena muchos árboles profundos sobre **muestras bootstrap** (sorteos con reemplazo del mismo tamaño) y promedia sus votos. Como cada árbol se equivoca de manera distinta, el promedio reduce la **varianza**. El **bosque aleatorio** agrega otra fuente de diversidad: en cada corte solo considera un subconjunto aleatorio de variables (`max_features`), lo que decorrelaciona los árboles y reduce más la varianza.

- **Error OOB:** cada muestra bootstrap deja fuera ≈ 37 % de los datos ($(1 - 1/n)^n \to e^{-1}$). Evaluar cada punto solo con los árboles que no lo vieron da una estimación tipo validación sin separar datos (`oob_score=True`). En credit-g: 0.757 OOB frente a 0.747 ± 0.017 por validación cruzada.
- **Más árboles no sobreajusta:** `n_estimators` se pone suficientemente grande (cientos); solo cuesta cómputo.
- **Importancia de variables:** la MDI (reducción de impureza) se calcula en entrenamiento y favorece a las variables con muchos valores posibles; la importancia **por permutación**, medida en datos no vistos, es más confiable (`permutation_importance`).

Boosting (referencia)

El boosting encadena aprendices débiles (árboles pequeños): cada uno se ajusta a lo que los anteriores no explican. En el *gradient boosting* con pérdida cuadrática, la etapa m ajusta un árbol h_m a los residuos y suma $F_m = F_{m-1} + \nu\, h_m$, con una tasa ν pequeña (*shrinkage*). Reduce sobre todo el **sesgo** y, a diferencia del bosque, **sí puede sobreajustar** con demasiadas etapas: se usa parada temprana. En scikit-learn, `HistGradientBoostingClassifier` es rápido, maneja faltantes y categóricas de forma nativa y trae `early_stopping`; lo usamos como referencia de desempeño.

```python
from sklearn.ensemble import (RandomForestClassifier,
                              HistGradientBoostingClassifier)

rf = RandomForestClassifier(n_estimators=500, max_features="sqrt",
                            oob_score=True, random_state=2026)
hgb = HistGradientBoostingClassifier(
    learning_rate=0.05, max_iter=500, early_stopping=True,
    categorical_features="from_dtype", random_state=2026)
```

> **Practica esto →** **E2.4 · Nivel 2** (OOB, MDI frente a permutación y parada temprana) · **A2.7 bosque** · **A2.8 boosting**

§9 Comparar modelos con honestidad

Elegir entre modelos es una decisión estadística: las diferencias deben ser mayores que el ruido de la estimación.

- **Sesgo y varianza:** el error esperado se descompone en sesgo², varianza y ruido irreducible (M1 §4). Los modelos simples (lineales, árboles poco profundos) tienen sesgo; los flexibles (árboles profundos, RBF con gamma alto) tienen varianza. Regularizar y promediar reducen varianza; agregar capacidad reduce sesgo.
- **Curvas de aprendizaje** (`learning_curve`): error de entrenamiento y validación según el tamaño de la muestra. Ambas altas y juntas → sesgo: más datos no ayudan, hace falta más capacidad. Brecha grande → varianza: más datos o más regularización.
- **Búsqueda de hiperparámetros sin fuga:** `GridSearchCV` o `RandomizedSearchCV` con el pipeline completo y solo con el entrenamiento. La prueba se usa una vez, al final.
- **Validación anidada:** el mejor puntaje de la malla es optimista, porque se eligió con esos mismos folds. La validación anidada mete la búsqueda dentro de cada fold externo. En E2.4, sobre ruido puro, el "mejor de la malla" reporta AUC 0.54 y la estimación anidada 0.49 (la verdad es 0.5).

- **Desbalance y costos:** métricas como PR-AUC y el costo esperado; umbral elegido por validación cruzada. En credit-g (costo 5:1) el bosque con umbral 0.17 cuesta 0.56 por solicitud en la prueba; con umbral 0.5 cuesta 1.00, peor que rechazar a todos (0.70).
- **Reportar media ± DE** entre folds. Si dos modelos quedan dentro de una desviación estándar, prefiere el más simple, explicable o barato.
- **Seguimiento de experimentos (BC7):** registra cada corrida con sus parámetros, métricas, versión de datos y modelo en **MLflow**. Es la base de la reproducibilidad de la Tarea 2.

Python

```python
import mlflow
from sklearn.model_selection import GridSearchCV

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("T2-jm1")
with mlflow.start_run(run_name="bosque"):
    busq = GridSearchCV(pipe, malla, cv=cv, scoring="average_precision")
    busq.fit(X_train, y_train)
    mlflow.log_params(busq.best_params_)
    mlflow.log_metrics({"pr_auc_cv": busq.best_score_})
# Explorar: mlflow ui --backend-store-uri sqlite:///mlflow.db
```

⚠ **Errores frecuentes al comparar**

Elegir hiperparámetros o el umbral con la prueba; escalar o seleccionar variables fuera del pipeline; declarar un ganador por milésimas sin mirar la desviación; comparar modelos con particiones distintas; reportar la exactitud con clases desbalanceadas.

Practica esto → **E2.4 · Niveles 3 y 4** (laboratorio del sábado: 5 modelos en MLflow y validación anidada) · **A2.9 sesgo y varianza** · **A2.10 curvas de aprendizaje** · **Tarea 2**

§10 Autoevaluación y lecturas

▼ **1.** Tu descenso de gradiente diverge con $\eta = 0.1$ en un problema y converge en otro. ¿Qué revisarías primero?

La escala de las variables. Sin estandarizar, el hessiano tiene un valor propio grande y la tasa estable $2/\lambda_{\max}$ se vuelve diminuta. Estandarizar dentro del pipeline lo resuelve.

▼ 2. **Lasso dejó en cero la variable `AveBedrms`. ¿Significa que no influye en el precio?**

No necesariamente. Con variables correlacionadas (`AveRooms` y `AveBedrms`), Lasso conserva una y anula la otra de forma algo arbitraria. Un coeficiente en cero dice que la variable no aporta *dado el resto* y con ese α.

▼ 3. **Dos modelos tienen la misma AUC, pero uno tiene Brier 0.085 y el otro 0.185. ¿Cuál usarías para priorizar llamadas por valor esperado?**

El de Brier 0.085: ordenan igual, pero sus probabilidades están calibradas y la regla $p > c/g$ funciona. El otro habría que calibrarlo primero (Platt o isotónica).

▼ 4. **Una SVM RBF tiene 100 % de exactitud en entrenamiento y 81 % en validación. ¿Hacia dónde mueves C y gamma?**

Hacia valores más pequeños: menos gamma suaviza la frontera y menos C tolera más violaciones del margen. Se busca en una malla logarítmica con validación cruzada.

▼ 5. **Un KNN con k = 1 tiene 100 % de exactitud en entrenamiento y 86 % en validación, con variables en unidades muy distintas. ¿Qué cambias primero?**

Primero, escalar las variables dentro del pipeline: sin escalar, la variable de unidad mayor decide sola quiénes son los vecinos. Después, subir k y elegirlo por validación cruzada: k = 1 memoriza el entrenamiento (varianza alta).

▼ 6. **Tus curvas de aprendizaje muestran error de entrenamiento 19 % y de validación 20 %, casi planas desde n = 200. ¿Conseguir más datos es buena inversión?**

No. Es sesgo alto: el modelo no tiene capacidad. Conviene un modelo más flexible (árboles, bosque, kernel) o mejores variables.

▼ 7. **¿Por qué el error OOB de un bosque se parece al de validación cruzada?**

Porque cada punto se evalúa solo con los árboles que no lo usaron para entrenar (≈ 37 % de ellos): para ese punto, esos árboles son un modelo que no lo vio.

Lecturas de la semana

- **BB3** · James et al. (2023), ISLP: caps. 2 (Statistical Learning: el clasificador KNN), 3 (Linear Regression, con la comparación frente a KNN), 4 (Classification), 6 (Linear Model Selection and Regularization), 8 (Tree-Based Methods) y 9 (Support Vector Machines).
- **BB4** · Raschka et al. (2022): caps. 3 (clasificadores con scikit-learn), 6 (evaluación y ajuste de hiperparámetros) y 7 (ensambles).
- **BB1** · Géron (2025): capítulos de entrenamiento de modelos lineales, SVM, árboles de decisión y ensambles.

- **BB5** · scikit-learn User Guide: Linear models, SVM, Nearest Neighbors, Decision Trees, Ensembles, Probability calibration y Tuning the hyper-parameters.
- **BC3** · scikit-learn: Common pitfalls and recommended practices (fuga al buscar hiperparámetros).
- **BC7** · MLflow Documentation: seguimiento de experimentos.